

[수업중 실습2-1]

Student(1to1) Client 를 **Vite** 기반 **ReactJS** 프로젝트로 작성하기

```
npm create vite@latest student-client-react -- --template react
```

```
cd student-client-react
```

```
npm install
```

React 컴포넌트 설계 원칙 및 가이드

React 컴포넌트를 분리하여 작성할 때 적용한 주요 원칙들은 다음과 같습니다:

1. 단일 책임 원칙 (Single Responsibility Principle)

적용 방법:

- 각 컴포넌트는 한 가지 일만 담당합니다.
- 예: **StudentTable**은 학생 목록 표시만, **StudentForm**은 학생 정보 입력만 담당합니다.

장점:

- 코드 가독성 향상
- 재사용성 증가
- 유지보수 용이성

2. 관심사 분리 (Separation of Concerns)

적용 방법:

- UI 표현과 데이터 처리 로직을 분리합니다.
- 상태 관리는 상위 컴포넌트에서, 데이터 표시는 하위 컴포넌트에서 담당합니다.

예시:

- **StudentManager**: 데이터 관리, API 호출, 상태 저장
- **StudentTable**: 데이터 표시만 담당, 직접 API 호출하지 않음

3. 컴포넌트 계층구조 (Component Hierarchy)

적용 방법:

- 컴포넌트를 논리적 계층으로 구성합니다.
- 부모-자식 관계를 명확히 구조화합니다.

App



4. 프레젠테이션 vs 컨테이너 컴포넌트 패턴

적용 방법:

- Container 컴포넌트: 상태 관리, 데이터 처리 (**StudentManager**)
- Presentational 컴포넌트: UI 표현만 담당 (**StudentTable**, **Modal** 등)

장점:

- 로직과 UI의 명확한 분리
- 재사용성 향상
- 테스트 용이성

5. 합성(Composition) 활용

적용 방법:

- 작은 컴포넌트를 조합하여 복잡한 UI 구성
- props를 통한 데이터 전달

```
<StudentManager>
  <StudentForm onSubmit={createStudent} />
  <StudentTable students={students} />
</StudentManager>
```

6. Props와 State 명확히 구분

적용 방법:

- Props: 부모로부터 받는 읽기 전용 데이터
- State: 컴포넌트 내부에서 관리하는 변경 가능 데이터

// 프로퍼티: 외부에서 전달받음

```
const StudentTable = ({ students, onEdit, onDelete }) => { ... }
```

// 상태: 내부에서 관리

```
const [formData, setFormData] = useState({ name: "", studentNumber: "" });
```

7. 재사용 가능한 유틸리티 함수 분리

적용 방법:

- 공통 로직은 별도 함수로 추출
- 여러 컴포넌트에서 사용되는 함수는 별도 파일로 분리

// StudentTable.jsx 내부

```
const renderCell = (content) => {
  if (!content || content === "-") return "-";
  return <div className="truncate" title={content}>{content}</div>;
};
```

8. 컴포넌트 크기 제한

적용 방법:

- 300줄 이상 커지면 더 작은 컴포넌트로 분할 고려
- 한 파일에 한 컴포넌트만 정의

장점:

- 코드 가독성 향상
- 유지보수 용이
- 협업 시 충돌 감소

10. 에러 처리 및 유효성 검사 통합

적용 방법:

- 각 컴포넌트에 적절한 에러 처리 로직 포함
- 폼 컴포넌트에 유효성 검사 통합

```
// StudentForm 내부
const validateForm = () => {
  const errors = {};
  if (!formData.name) errors.name = "이름을 입력해주세요";
  // ...
  return errors;
};
```

추가 가이드라인

1. 명명 규칙 일관성
 - 컴포넌트: PascalCase (StudentTable)
 - 함수/변수: camelCase (handleSubmit)
 - 상수: UPPER_SNAKE_CASE (API_BASE_URL)
2. **CSS** 분리 및 모듈화
 - 컴포넌트별 CSS 파일 분리
 - CSS 모듈 또는 스타일드 컴포넌트 고려
3. 성능 최적화
 - React.memo, useCallback, useMemo 적절히 활용
 - 불필요한 리렌더링 방지

React CSS 파일 구조 및 각 파일의 역할

프로젝트에서 사용된 세 가지 CSS 파일(index.css, App.css, StudentManager.css)은 각각 다른 범위와 목적을 가지고 있습니다. 이들의 역할과 관계를 설명해드리겠습니다.

1. index.css

역할

- 전역 스타일 정의: 애플리케이션 전체에 적용되는 가장 기본적인 스타일
- CSS 리셋/노멀라이즈: 브라우저 간 일관성을 유지하기 위한 기본 스타일 초기화
- 루트 변수 정의: CSS 변수(--primary-color 등)와 같은 전역 변수

특징

- main.jsx에서 가장 먼저 임포트되어 모든 컴포넌트에 영향
- 페이지 전체의 기본 레이아웃과 색상 체계 설정
- 앱 전반에 걸쳐 일관된 디자인 시스템의 기초 제공

2. App.css

역할

- 애플리케이션 레벨 스타일: App의 주요 레이아웃 및 공통 컴포넌트 스타일
- 재사용 가능한 UI 요소: 버튼, 폼, 테이블 등 범용 컴포넌트 스타일
- 공통 유틸리티 클래스: 여러 컴포넌트에서 사용하는 공통 스타일 클래스

특징

- App.jsx에서 임포트 되어 애플리케이션의 기본 UI 프레임워크 제공
- 여러 컴포넌트에서 공유되는 스타일 정의
- 일관된 UI 요소를 위한 클래스 제공
- 애플리케이션의 전반적인 레이아웃과 스타일링 시스템 정의

3. StudentManager.css

역할

- 컴포넌트 특화 스타일: StudentManager 컴포넌트와 그 하위 요소만을 위한 스타일
- 기능별 세부 스타일링: 학생 관리 기능에 특화된 UI 요소의 스타일
- 지역 범위 스타일: 다른 컴포넌트와 충돌하지 않는 독립적인 스타일

특징

- StudentManager.jsx에서만 임포트되는 컴포넌트 특화 스타일
- 특정 기능의 고유한 디자인 요구사항을 충족
- 다른 컴포넌트와 스타일 충돌 방지
- 컴포넌트의 독립성과 재사용성 향상

세 가지 CSS 파일의 관계 및 계층 구조

index.css (전역 스타일)



App.css (애플리케이션 레벨 공통 스타일)



StudentManager.css (컴포넌트 특화 스타일)

스타일 우선순위

1. 가장 낮은 우선순위: index.css (전역 스타일)
2. 중간 우선순위: App.css (애플리케이션 공통 스타일)
3. 가장 높은 우선순위: StudentManager.css (컴포넌트 특화 스타일)

이는 CSS의 일반적인 특성인 "구체성(Specificity)"과 "캐스케이딩(Cascading)" 원칙에 따른 것입니다.

1. 캐스케이딩 (Cascading)

"계단식으로 흐르는" 원칙으로, CSS 규칙이 여러 곳에서 정의될 때 어떤 스타일이 최종적으로 적용될지 결정하는 방식입니다.

주로 3가지 기준으로 우선순위가 정해집니다:

1) 출처 (Source)

사용자 스타일 (브라우저 확장 등) > 제작자 스타일 (개발자가 작성한 CSS) > 브라우저 기본 스타일

2) 중요도 (important)

!important가 붙은 스타일이 가장 우선시됩니다. (남용하면 유지보수가 어려워요!)

3) 선택자 순서

나중에 선언된 스타일이 이전 스타일을 덮어씁니다.

```
/* 예시: 같은 선택자라면 나중에 오는 스타일이 적용됩니다. */
p { color: blue; } /* 적용 안 됨 */
p { color: red; } /* 최종 적용 */
```

2. 구체성 (Specificity)

선택자의 구체적인 정도에 따라 우선순위가 결정됩니다.

점수 계산식으로 이해하면 쉬워요 (점수가 높을수록 우선순위 ↑):

선택자 유형	점수 (예시)
인라인 스타일	1000 (<p style="color: red">)
ID 선택자 (#id)	100 (#header)
클래스/속성/가상 선택자	10 (.btn, :hover)
태그/가상 요소 선택자	1 (p, ::before)
전체 선택자 (*)	0 (우선순위 영향 없음)

```
/* 예시: 구체성이 높은 선택자가 이깁니다. */
#main .text { color: blue; } /* 100 + 10 = 110점 */
div p { color: red; } /* 1 + 1 = 2점 → blue 적용 */
```

1) src/index.css

<https://gist.github.com/mysoyul/eea724ae949b30e703e316093a765832>

2) src/App.css

<https://gist.github.com/mysoyul/ea30d7dc512cdb8e6ebfa8d16b7b6a21>

src아래에 components 폴더를 생성합니다.

3) src/components/StudentManager.css

<https://gist.github.com/mysoyul/97c9b538c8f00a66e9bf23c9bea5b8bc>

4) src/components/common/LoadingSpinner.jsx

<https://gist.github.com/mysoyul/7a492ee0d3bc687625ceffaec5faf458>

5) src/components/common/MessageContainer.jsx

<https://gist.github.com/mysoyul/48ea6776125676dee19311080a11ce37>

6) src/components/StudentForm.jsx

<https://gist.github.com/mysoyul/fb8e9847aff46ef202c5d87c5262ac36>

7) src/components/StudentSearch.jsx

<https://gist.github.com/mysoyul/10c7601bb440cdf18e47a6e23ef1d68b>

8) src/components/StudentTable.jsx

<https://gist.github.com/mysoyul/5923d4717bdfd759f5402877b3f7efc>

9) src/components/StudentDetailModal.jsx

<https://gist.github.com/mysoyul/68f7cb34a5404c5657d195c3910fb436>

10) src/components/StudentEditModal.jsx

<https://gist.github.com/mysoyul/704762b645e132a9d038cb3b2a00221f>

11) .env.development

```
# API URL  
VITE_API_URL=http://localhost:8080/api
```

12) src/components/StudentManager.jsx

<https://gist.github.com/mysoyul/facd71ca6ba91efbde233b90d3fd4e89>

13) src/App.jsx

<https://gist.github.com/mysoyul/657160076981e3eb98ecda747c25b3c3>

[실습2-2]

Book(1to1) Client 를 **Vite** 기반 **ReactJS** 프로젝트로 작성하기

```
npm create vite@latest book-client-react -- --template react
```

```
cd book-client-react
```

```
npm install
```

프로젝트 구조

book-client-react/

```
├── public/
├── src/
│   ├── components/
│   │   ├── BookManager.jsx      # 메인 컴포넌트
│   │   ├── BookManager.css     # 메인 컴포넌트 스타일
│   │   ├── BookForm.jsx        # 도서 등록 Form
│   │   ├── BookSearch.jsx      # 도서 검색
│   │   ├── BookTable.jsx       # 도서 목록 테이블
│   │   ├── BookDetailModal.jsx # 도서 상세 Modal
│   │   ├── BookEditModal.jsx   # 도서 수정 Modal
│   │   └── common/
│   │       ├── LoadingSpinner.jsx # 로딩 스피너
│   │       └── MessageContainer.jsx # 메시지 컨테이너
│   ├── App.jsx                 # 애플리케이션 진입점
│   ├── App.css                 # 공통 스타일
│   └── main.jsx                 # React 렌더링 설정
├── .env.development            # 개발 환경 변수
├── package.json
└── vite.config.js
```

주요 기능

1. 도서 관리
 - 도서 추가, 조회, 수정, 삭제 (CRUD)
 - 도서 상세 정보 관리
2. 검색 기능
 - 제목, 저자, 출판사별 검색
3. UI 기능
 - 모달 다이얼로그 (상세 보기, 수정)
 - 로딩 상태 표시
 - 유효성 검사
 - 응답 메시지 알림

API 연결

이 애플리케이션은 백엔드 API와 통신합니다. API 엔드포인트는 `.env.development` 파일에 구성되어 있습니다:

VITE_API_URL=<http://localhost:8080/api>

컴포넌트 설명

BookManager

- 애플리케이션의 메인 컴포넌트
- 상태 관리 및 API 호출 관리
- 하위 컴포넌트 조합

BookForm

- 새 도서 등록 폼
- 기본 정보 및 상세 정보 입력
- 유효성 검사

BookSearch

- 도서 검색 기능
- 제목, 저자, 출판사별 검색
- 전체 목록 보기

BookTable

- 도서 목록 표시
- 액션 버튼 (상세, 수정, 삭제)

BookDetailModal

- 도서 상세 정보 모달
- 기본 정보 및 상세 정보 표시
- 표지 이미지 표시

BookEditModal

- 도서 정보 수정 모달
- 모든 필드 편집 가능
- 유효성 검사

LoadingSpinner

- 로딩 상태 표시

MessageContainer

- 성공/오류 메시지 표시

1) src/index.css

<https://gist.github.com/mysoyul/7b1c11e1fb85872d2532ddf6f1d86df3>

2) src/App.css

<https://gist.github.com/mysoyul/2f460b6043d50628a08a0541808ec408>

src아래에 components 폴더를 생성합니다.

3) src/components/BookManager.css

<https://gist.github.com/mysoyul/e51967a7436df6684a7ba76c411d5c33>

4) src/components/common/LoadingSpinner.jsx

<https://gist.github.com/mysoyul/7a492ee0d3bc687625ceffaec5faf458>

5) src/components/common/MessageContainer.jsx

<https://gist.github.com/mysoyul/48ea6776125676dee19311080a11ce37>

6) src/components/BookForm.jsx

7) src/components/BookSearch.jsx

8) src/components/BookTable.jsx

9) src/components/BookDetailModal.jsx

10) src/components/BookEditModal.jsx

11) .env.development

API URL

VITE_API_URL=<http://localhost:8080/api>

12) src/components/BookManager.jsx

13) src/App.jsx

